

ChatGPT 网页到 Codex 的 Responses API 实验桥接器

3099404236^{1*}

¹Independent developer

本报告记录一个 Windows 本地实验桥接器：它接收 Codex 发出的 Responses API 请求，通过可见浏览器复用已登录的 ChatGPT 网页对话，再把网页回答转换为文本、函数调用或原生补丁事件。实现重点不是把网页伪装成完整官方 API，而是验证一条短期可用的代理链路能否维持同一对话上下文，并支持“读取文件、修改代码、运行测试、根据结果继续修正”的多轮循环。公开版本包含实现、配置、启动脚本与 30 项自动化测试，不包含浏览器登录资料、真实对话、运行日志或私用密钥。

Codex | Responses API | ChatGPT Web | browser automation | agent tools

 <https://github.com/3099404236>

背景与目标

Codex 的代理能力并不只来自模型本身。外层客户端负责保存任务状态、向模型声明工具、在本机执行命令和补丁，再把工具结果送回模型。由此可以提出一个实验问题：如果把模型推理链路临时替换为已经登录的 ChatGPT 网页，外层代理循环是否仍能工作？

本项目的目标是建立一个最小、可观察、可撤销的本地桥接层。它只服务于短期试验，不声称与 OpenAI 官方 Responses API 完全兼容，也不把 ChatGPT 订阅变成 API 额度。成功标准是：

- Codex 专用窗口的模型请求不经过原来的模型 provider；
- 同一个任务尽量留在同一个 ChatGPT 网页对话中；
- 网页可以返回最终文本，也可以要求 Codex 执行受支持的本机工具；
- 工具结果能够进入下一轮，直到任务完成；
- 登录数据、会话数据库和运行日志不进入公开代码。

系统架构

桥接器使用 prompt_cache_key 和请求上下文构造稳定的任务键。一个任务首次出现时建立网页对话；后续工具轮只发送新增的工具输出，并剔除 Codex 回传的重复网页回答。默认每个网页对话最多承载 24 次回答或累计 180000 个输入输出字符，超过阈值后创建新对话，并以 Codex 当前完整上下文衔接。内存中最多跟踪 8 个任务。

多轮工具循环

一次完整代理循环按以下顺序进行：

1. Codex 把用户任务、可用工具和工作目录发送到本地端点。
2. 桥接器把必要指令与新输入写入 ChatGPT 网页。
3. 网页模型返回最终文本，或返回严格的 YAML/JSON 工具请求。
4. 桥接器把工具请求转换为 Responses 的 function_call 或 custom_tool_call。
5. Codex 在本机执行工具，并把 function_call_output 或 custom_tool_call_output 放进下一次请求。
6. 桥接器在同一网页对话中追加工具结果，模型据此继续判断，直至返回最终回答。

这解释了为什么单次用户输入可以产生多轮文件操作：循环控制者是 Codex，网页模型每一轮只决定“下一步工具”或“任务已经完成”。

工具协议与补丁策略

网页输出不是可信的本机命令源，因此桥接器只接受白名单工具。工具即使报告退出码 0，只要输出含 traceback、语法错误或补丁失败特征，也会被标记为失

层	组件	职责
1	Codex 专用窗口	组织代理循环，声明工具，在本机执行 shell_command 与 apply_patch。
2	本地 Responses 端点	监听 localhost:8766，接收 JSON 或 SSE 请求，维护任务级会话状态。
3	浏览器驱动	复用可见 Chrome 或 Edge 进程，在已登录的 ChatGPT 页面中发送增量内容并等待生成结束。
4	协议转换	把网页文本解析成最终回答、函数调用或自定义补丁调用，再包装成 Responses 事件。

表 1 四层结构把本机工具执行与网页模型推理解耦。

能力	返回形式	实现边界
命令执行	function_call	允许 shell 与 shell_command；实际命令由 Codex 客户端执行。
文本补丁	custom_tool_call	优先使用原生 apply_patch，避免 PowerShell 字符串转义破坏补丁。
补丁后备	shell_command	仅在原生补丁被 Windows 运行时拒绝后启用；路径必须位于当前工作目录内，并拒绝 .git 元数据。
计划更新	function_call	允许 update_plan，用于长任务的阶段状态。
最终回答	message	没有工具请求时作为完成结果返回。

表 2 公开版本的工具类型和执行边界。

败并交还模型重新判断。客户端断开连接时，当前网页生成会被取消，避免后续请求一直堵塞。

完成检测与错误恢复

网页界面没有稳定的 API 完成事件。实现综合使用“新回答节点或文本变化、文本稳定、生成停止”三个信号判断回答结束。浏览器进程和登录配置在桥接器运行期间保持复用；切换 Codex 任务时，桥接器回到该任务已经记录的网页对话 URL。

如果页面出现通用错误卡片，默认只执行一次 Retry 或 Regenerate。项目明确不处理或绕过 验证码、网站用量限制、登录限制和风控。页面结构变化仍可能导致选择器失效，此时需要根据 本地调试截图和 HTML 更新浏览器驱动。

安全与隐私边界

公开仓库只包含程序、配置示例、测试和文档。ChatGPT 浏览器配置、Codex 会话数据库、请求审计日志、截图、缓存、node_modules 和本机私用 ZIP 均被排除。

专用 Codex 配置使用独立 CODEX_HOME，不会覆盖全局配置。当前实验为了让工具 直接操作文件而设置 danger-full-access；这意味着该进程可以访问工作区外文件和网络，只应在信任网页对话与任务内容时运行。关闭专用窗口即可结束该进程权限。

配置还关闭插件市场、Apps、多代理和内置网页搜索。原因是本地端点只实现最小 Responses 协议，并没有 OpenAI 托管的 web_search 后端。ChatGPT 网页自身

虽然可能提供搜索，但当前公开版本尚未把搜索动作和来源引用转换成结构化工具事件。

验证结果

2026-07-15 在 Windows 本地执行完整 Node.js 测试套件，结果为 30 项通过、0 项失败，一次运行耗时约 8 秒。覆盖范围见 表 3。

测试只能证明协议与控制流在当前实现下成立，不能保证 ChatGPT 网页长期保持相同 DOM、按钮名称或服务行为。真实登录页面仍需要人工观察。

复现步骤

公开代码位于 3099404236/chatgpt-web-codex-bridge。在 Windows PowerShell 中：

```
cd code
npm install
npm test
.\scripts\open-codex-web-window.ps1
```

首次运行需要在弹出的可见浏览器中手动登录 ChatGPT。之后在标题为 Codex-Web 的终端窗口中输入任务。健康检查、直接 Responses 请求和全部环境变量见代码目录的 README。

局限与结论

主要局限是网页界面可能变化、结构化工具请求偶尔需要重试，而且当前没有结构化网页搜索、图像输入或完整官方 Responses 工具生态。ChatGPT 网页与 OpenAI API 仍是不同产品面，本项目不改变其授权、计费或使用边界。30 项测试说明，只要 Codex 保留外层代理循环，再用可靠协议转换网页文本和工具请求，网页模型就能参与连续的“读取、修改、验证、再修改”工作流。

测试组	状态	主要覆盖
Responses 转换	通过	普通 JSON、SSE 起止序列、函数调用和自定义工具事件。
多轮会话	通过	同对话增量续轮、工具结果回传、阈值轮换与请求取消。
工具协议	通过	YAML/JSON 解析、Windows 路径与引号修复、工具白名单。
补丁安全	通过	新增、修改、移动、删除以及工作区外路径拒绝。
网页驱动	通过	使用测试页面验证输入、生成等待和错误重试。

表 3 自动化测试覆盖的主要风险面。